Project: Optimizing a CNN Model for Animal Image Classification












Summary :

I. Introduction
   - Project Background
   - Goals and Expectations

II. Model Choices and Evolution
   - Starting with ResNet-18
   - Transition to ResNet-50
   - Experiments with ResNet-34
   - Returning to ResNet-18 and Exploring EfficientNetB0

III. Optimization Strategies
   - Training Dataset Splitting
   - Hyperparameter Tuning
   - Managing GPU Limitations
   - Data Augmentation

IV. Training and Evaluation Methodology
   - Training Setup
   - Ongoing Evaluation and Validation
   - Using Test Data

V. Performance Analysis
   - Loss and Accuracy Graphs
   - Hyperparameter Impact
   - Confusion Matrix and Error Analysis

VI. Challenges and Limitations
   - Resource Management
   - Overfitting Issues

VII. Conclusion and Outlook
   - Summary of Findings
   - Final Thoughts and Future Steps

Goal and Expectation :

Embarking on this project, my primary objective is to achieve a deep understanding of the intricacies involved in training a neural network for image classification. While I am currently in the exploratory phase, not entirely sure of the maximum achievable accuracy, I am setting my sights on an ambitious yet attainable goal. Aiming for a minimum accuracy of 80% seems both challenging and feasible, though I am aware of the complexities involved in reaching such a benchmark. The ideal target, however, would be to surpass 90% accuracy, acknowledging that this might be an extremely difficult feat.

This project serves as a platform for me to delve into the realm of Artificial Intelligence and machine learning, particularly in understanding and mastering the art of training models. It offers an opportunity to not just apply theoretical knowledge but also to innovate and experiment. Through this journey, I anticipate encountering various challenges and learning opportunities that will enhance my comprehension of model training dynamics and their practical applications. Ultimately, the experience gained from this project will be instrumental in broadening my horizons in AI and contributing to my growth as a machine learning enthusiast.

Context and Evolution of the Project :

- **Starting with ResNet-18:** The project began with a ResNet-18 model. However, initial challenges in understanding training and evaluation led me to explore more complex models.
- **Experimenting with ResNet-50:** I significantly increased Dropout (up to 0.8), Weight Decay (up to 1e-3), and decreased the Learning Rate (down to 0.00025) to combat overfitting. However, test accuracy plateaued despite improvements during training.
- **Transitioning to ResNet-34:** Seeking a balance between capacity and efficiency, I moved to ResNet-34, hoping to reduce overfitting while maintaining good performance.
- **Hyperparameter Strategies:** For the fully connected last layer, I increased hyperparameters like the learning rate by 10 times, considering this layer needed more extensive training compared to the pre-trained layers.
- **GPU Resource Limitations:** Faced with limited GPU access on Google Collaboratory and lengthy training times, I am considering a return to ResNet-18 for faster training.
- **Returning to ResNet-18:** Hoping to achieve satisfactory accuracy while overcoming hardware resource constraints.
- **In a need of make a better model, exploring EfficientNetB0 :** Opting for EfficientNetB0, the least resource-intensive among the new models reputed for higher accuracy.

Technical Strategies and Innovations:

1. **Training Database Division:** Implemented a program to divide Trainloaders into several folders, allowing the model to be saved at different stages and to resume training where it left off.

```
for fold, train_indices in enumerate(fold_indices):
  if fold + 1 < start_fold:
    continue  # Pass to next iteration

  print(f"Fold {fold + 1} / {num_folds} ou {num_folds + 1}")
  # Calculate index of start,end for each folds
  start_index = fold * fold_size
  end_index = start_index + fold_size if fold < num_folds - 1 else start_index + fold_size + remainder
  print(f"Start Index: {start_index}, End Index: {end_index}")
  print(f"Number of images in this fold: {len(train_indices)}")

  print(f"Folder number : {fold + 1}")
  train_subset = Subset(dataset, train_indices)
  train_loader = DataLoader(train_subset, batch_size=colab_args.batch_size, shuffle=True)

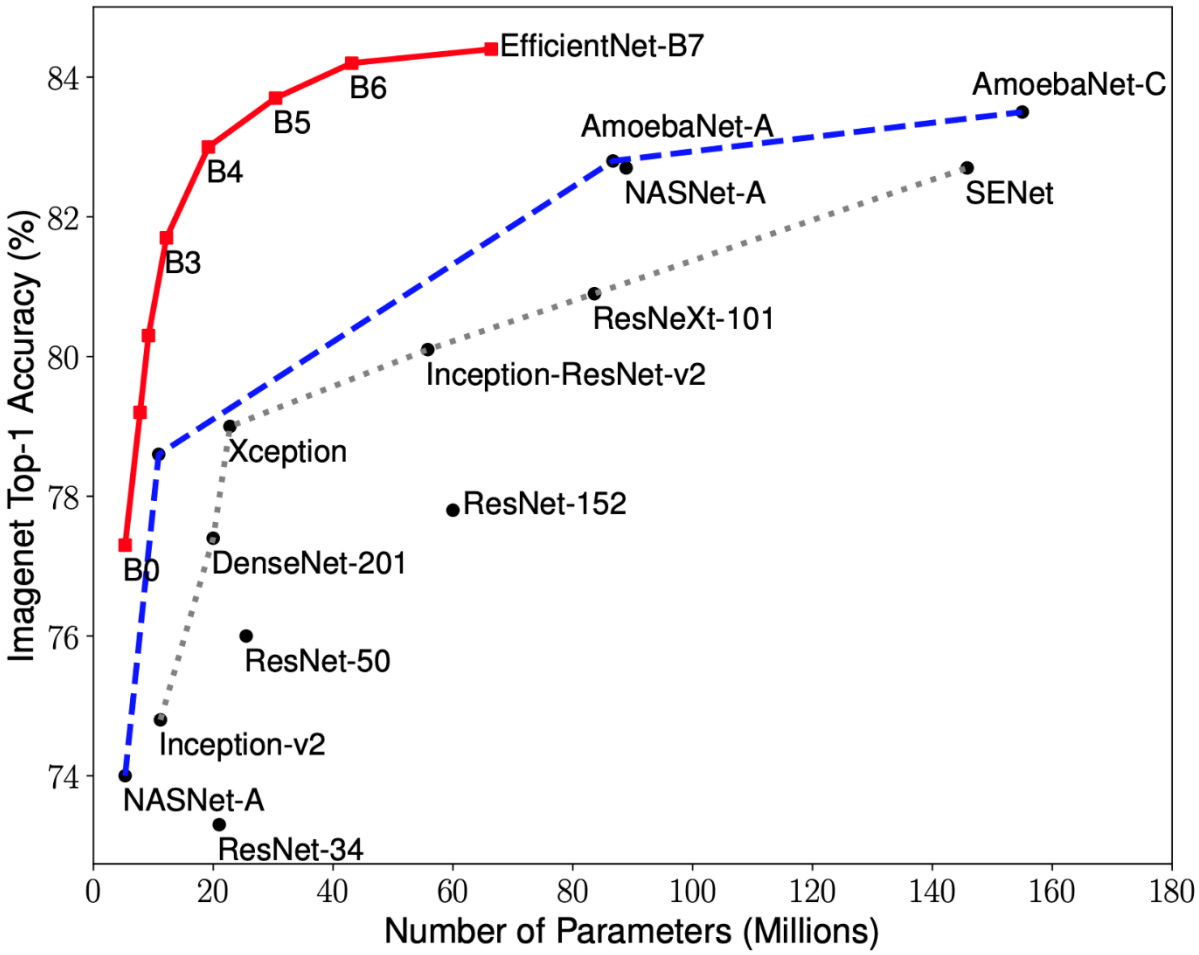
  #Training
  train(colab_args, model, train_loader, class_names, fold, colab_args_test, test_loader, optimizer)
```

2. **Using Pre-Trained Models:** To improve accuracy, I used various pre-trained models, including EfficientNetB0, and ResNet-18, ResNet-34, and ResNet-50 with ImageNet 1K. These models provided a solid starting point for image classification thanks to their extensive prior training.

ResNet18_Weights.IMAGENET1K_V1	69.758	89.078	11.7M	1.81	link
ResNet34_Weights.IMAGENET1K_V1	73.314	91.42	21.8M	3.66	link
ResNet50_Weights.IMAGENET1K_V1	76.13	92.862	25.6M	4.09	link
EfficientNet_B0_Weights.IMAGENET1K_V1	77.692	93.532	5.3M	0.39	link
EfficientNet_B1_Weights.IMAGENET1K_V1	78.642	94.186	7.8M	0.69	link
EfficientNet_B1_Weights.IMAGENET1K_V2	79.838	94.934	7.8M	0.69	link
EfficientNet_B2_Weights.IMAGENET1K_V1	80.608	95.31	9.1M	1.09	link
EfficientNet_B3_Weights.IMAGENET1K_V1	82.008	96.054	12.2M	1.83	link
EfficientNet_B4_Weights.IMAGENET1K_V1	83.384	96.594	19.3M	4.39	link
EfficientNet_B5_Weights.IMAGENET1K_V1	83.444	96.628	30.4M	10.27	link
EfficientNet_B6_Weights.IMAGENET1K_V1	84.008	96.916	43.0M	19.07	link
EfficientNet_B7_Weights.IMAGENET1K_V1	84.122	96.908	66.3M	37.75	link

EfficientNet models are the new models developped by Google, they are more precise and have less parameters, I choose the B0 due to the GPU restrictions.

This is a comparaison of popular models :



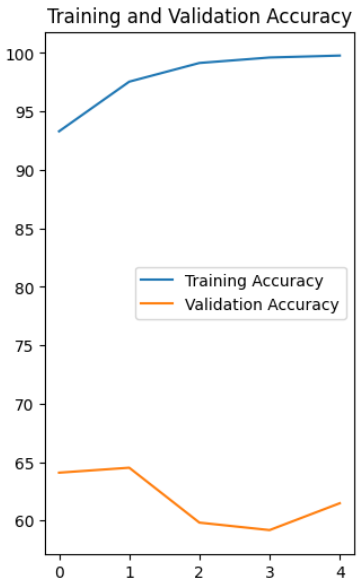
3. **Training Optimization:** Employed conditional prints to monitor images that exceed a certain loss function threshold, helping to assess overall model performance. Temporarily removed prints to speed up training.
4. **Continuous Performance Evaluation:** Conducted accuracy tests between different training folders to monitor the model's progress.

Graphs and Analysis Techniques:

1. **Loss and Accuracy Graphs:**
    - Display graphs of loss and accuracy over training and validation epochs to identify issues like overfitting or underfitting.
    - Compare graphs for different models or configurations to show the impact on performance.

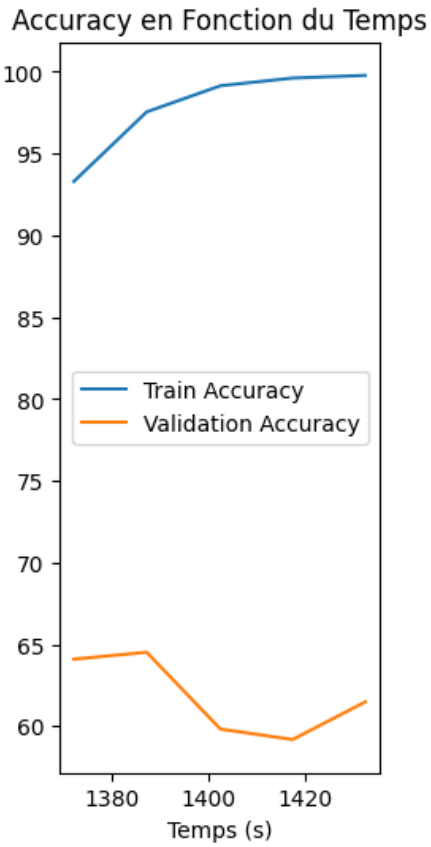
      Below it's my graph for Resnet18 model :

Training and Validation accuracy over epochs :

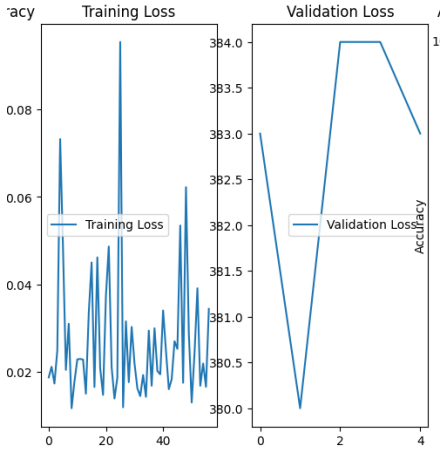


We can see a overfitting problem, with the resnet18, to solve this problem we must increase our dataset or choose a less complex model.

Accuracy over time :



Training loss over mini batchs and Validation Loss over epochs :

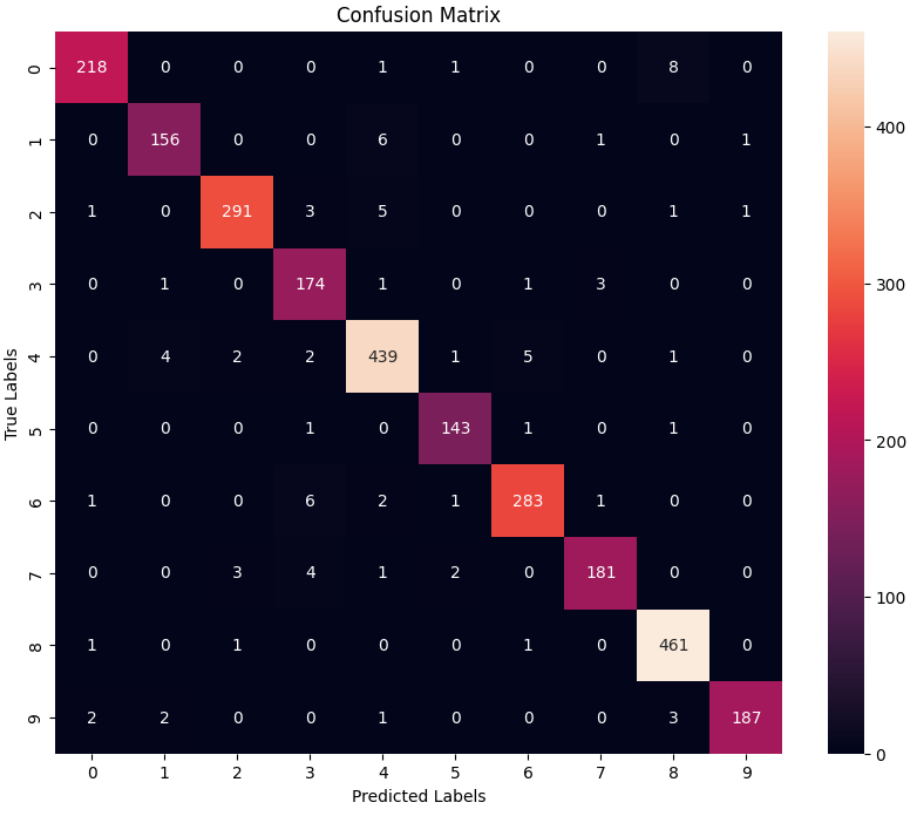


2. **Hyperparameter Graphs:**

- Test various key hyperparameter combinations and plot their impact on model performance.
- Consider creating a grid graph showing the effect of two varying hyperparameters, but I didn't have the ressources or time.

3. **Confusion Matrix:**

- A confusion matrix for test data can be highly informative, showing common error types made by the model. Below, there is the confusion matrix of my resnet18



For example class 0 is confond 8 times with class 8.

The class 0 (butterfly) is frequently confused with class 8 (spider), which is understandable since they may appear similar in certain aspects.
We can observe that classes 1, 2, and 6 have the highest number of misclassifications relative to their correct predictions.

This is reliable to the accuracy for each class :

EfficientNetB0 :

```
Accuracy globale: 69.00%
Accuracy for class 0 : 90.00%
Accuracy for class 1 : 50.00%
Accuracy for class 2 : 50.00%
Accuracy for class 3 : 70.00%
Accuracy for class 4 : 70.00%
Accuracy for class 5 : 80.00%
Accuracy for class 6 : 50.00%
Accuracy for class 7 : 60.00%
Accuracy for class 8 : 100.00%
Accuracy for class 9 : 70.00%
```

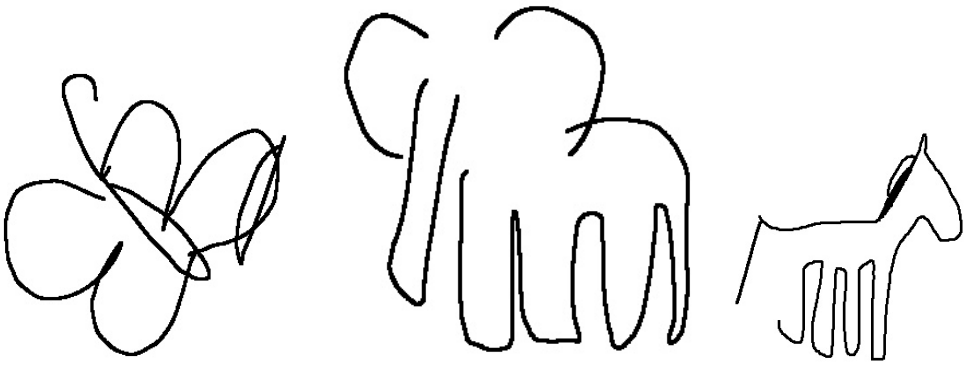
Resnet18 :

```
Accuracy globale: 71.00%
Accuracy for class 0 : 90.00%
Accuracy for class 1 : 40.00%
Accuracy for class 2 : 50.00%
Accuracy for class 3 : 70.00%
Accuracy for class 4 : 80.00%
Accuracy for class 5 : 90.00%
Accuracy for class 6 : 60.00%
Accuracy for class 7 : 60.00%
Accuracy for class 8 : 100.00%
Accuracy for class 9 : 70.00%
```

4. **Error Analysis:**

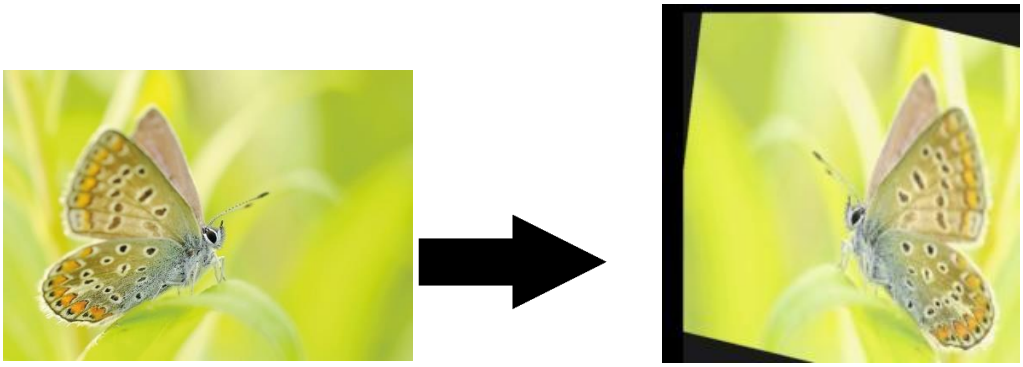
- We can analyse that models can make the same mistake, we can understand those mistakes.

  For example, every models will recognize drawing image as a spider. The model must be trained with more drawing if we want to solve this problem.



.

5. **Performance Graphs Over Training Time :**
    - In my CNN project, limited GPU access on Google Collaboratory has extended training times. Despite this, I've monitored model performance over available training time, which has been crucial for maximizing limited resources. Performance graphs have illustrated gradual improvements in accuracy, proving that strategic training can lead to progress even with resource constraints.
6. **Data Augmentation :** Modified existing images (flips, rotations, color changes) to artificially enrich the database and diversify training data.



7. **Model Validation and Testing:** In addition to training data, validation and test datasets are used to assess model accuracy and provide a reliable estimate of its performance.

Challenges and Limitations:

- **Resource Constraints:** Limited access to GPU and prolonged training durations.
- **Overfitting:** Struggled to achieve high test accuracy despite high training performance.

Current Outlook and Conclusion: The project continues with a return to ResNet-18, aiming for satisfactory accuracy while overcoming material resource constraints. The goal is to find a balance between model performance and efficient resource management.

At the project's conclusion, I had to significantly reduce my artificially expanded dataset due to time constraints, implying that with more time or access to GPUs with standard computational power, I could have potentially achieved higher accuracy.

My finally model is Resnet18 with 71% of Accuracy.
Thanks for reading

Source of help :

https://www.kaggle.com/code/francescolorenzo/96-fine-tuning-resnet34-with-pytorch/notebook

Deep learning classes
Image Net data

James Lemaitre